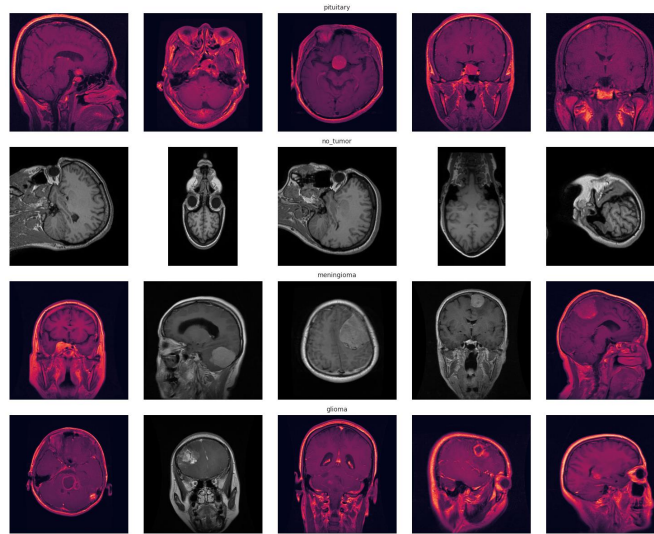


A Vision Transformer with FOTON-Based Orthogonalization for Robust Brain Tumor MRI Classification



```
import numpy as np
import pandas as pd
import os

root_path = '/kaggle/input/brisc2025/brisc2025/classification_task/train'

image_paths = []
labels = []

for label in os.listdir(root_path):
    label_path = os.path.join(root_path, label)
    if os.path.isdir(label_path):
        for img_file in os.listdir(label_path):
            if img_file.lower().endswith(('.png', '.jpg', '.jpeg')):
                image_paths.append(os.path.join(label_path, img_file))
                labels.append(label)

df = pd.DataFrame({'image_path': image_paths, 'label': labels})

df
```

	image_path	label
0	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
1	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
2	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
3	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
4	/kaggle/input/brisc2025/brisc2025/classificati...	pituitary
...	...	...
4995	/kaggle/input/brisc2025/brisc2025/classificati...	glioma
4996	/kaggle/input/brisc2025/brisc2025/classificati...	glioma
4997	/kaggle/input/brisc2025/brisc2025/classificati...	glioma
4998	/kaggle/input/brisc2025/brisc2025/classificati...	glioma
4999	/kaggle/input/brisc2025/brisc2025/classificati...	glioma

```

[5000 rows x 2 columns]

df.shape

(5000, 2)

df.columns

Index(['image_path', 'label'], dtype='object')

df.duplicated().sum()

0

df.isnull().sum()

image_path    0
label         0
dtype: int64

df.info()

<class 'pandas.core.frame.DataFrame'>
RangeIndex: 5000 entries, 0 to 4999
Data columns (total 2 columns):
 #   Column      Non-Null Count  Dtype
---  -
 0   image_path  5000 non-null   object
 1   label       5000 non-null   object
dtypes: object(2)
memory usage: 78.3+ KB

df['label'].unique()

array(['pituitary', 'no_tumor', 'meningioma', 'glioma'], dtype=object)

df['label'].value_counts()

label
pituitary    1457
meningioma   1329
glioma       1147
no_tumor     1067
Name: count, dtype: int64

import seaborn as sns
import matplotlib.pyplot as plt

sns.set_style("whitegrid")

fig, ax = plt.subplots(figsize=(8, 6))
sns.countplot(data=df, x="label", palette="viridis", ax=ax)

ax.set_title("Distribution of Tumor Types", fontsize=14, fontweight='bold')

```

```

ax.set_xlabel("Tumor Type", fontsize=12)
ax.set_ylabel("Count", fontsize=12)

for p in ax.patches:
    ax.annotate(f'{int(p.get_height())}',
                (p.get_x() + p.get_width() / 2., p.get_height()),
                ha='center', va='bottom', fontsize=11, color='black',
                xytext=(0, 5), textcoords='offset points')

plt.show()

label_counts = df["label"].value_counts()

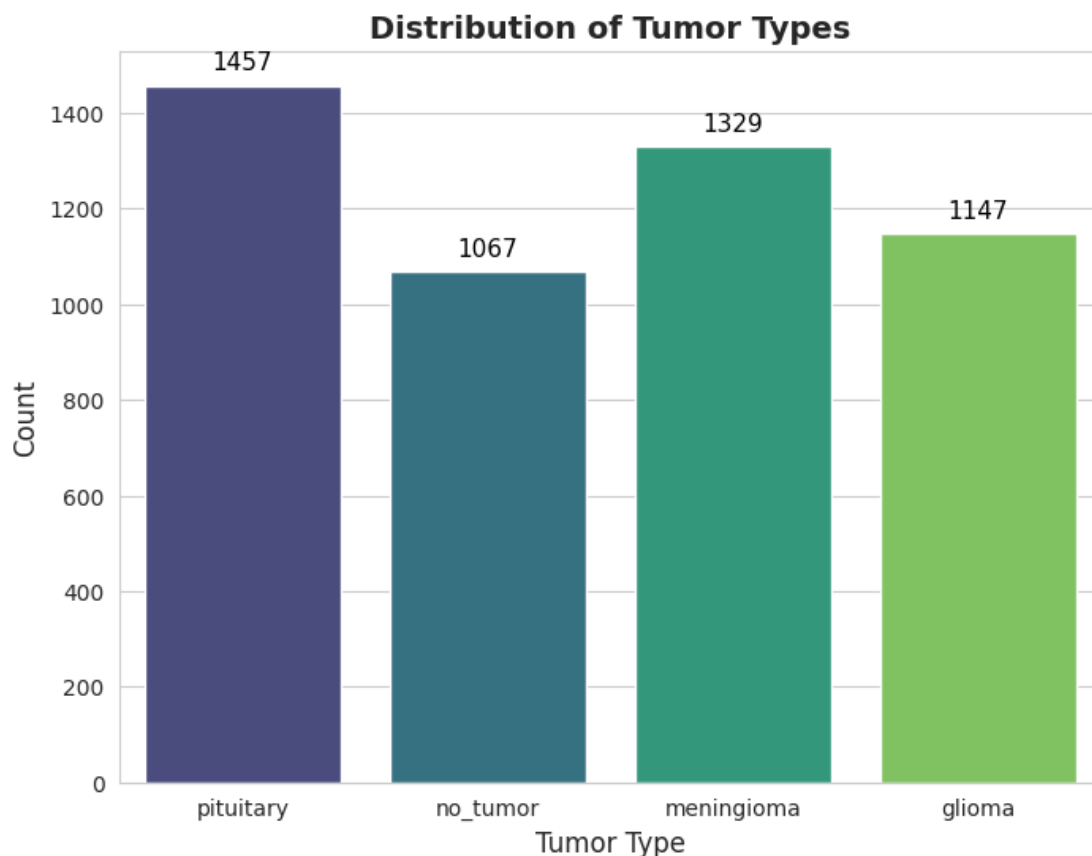
fig, ax = plt.subplots(figsize=(10, 8))
colors = sns.color_palette("viridis", len(label_counts))

ax.pie(label_counts, labels=label_counts.index, autopct='%1.1f%%',
        startangle=140, colors=colors, textprops={'fontsize': 8, 'weight':
'bold'},
        wedgeprops={'edgecolor': 'black', 'linewidth': 1})

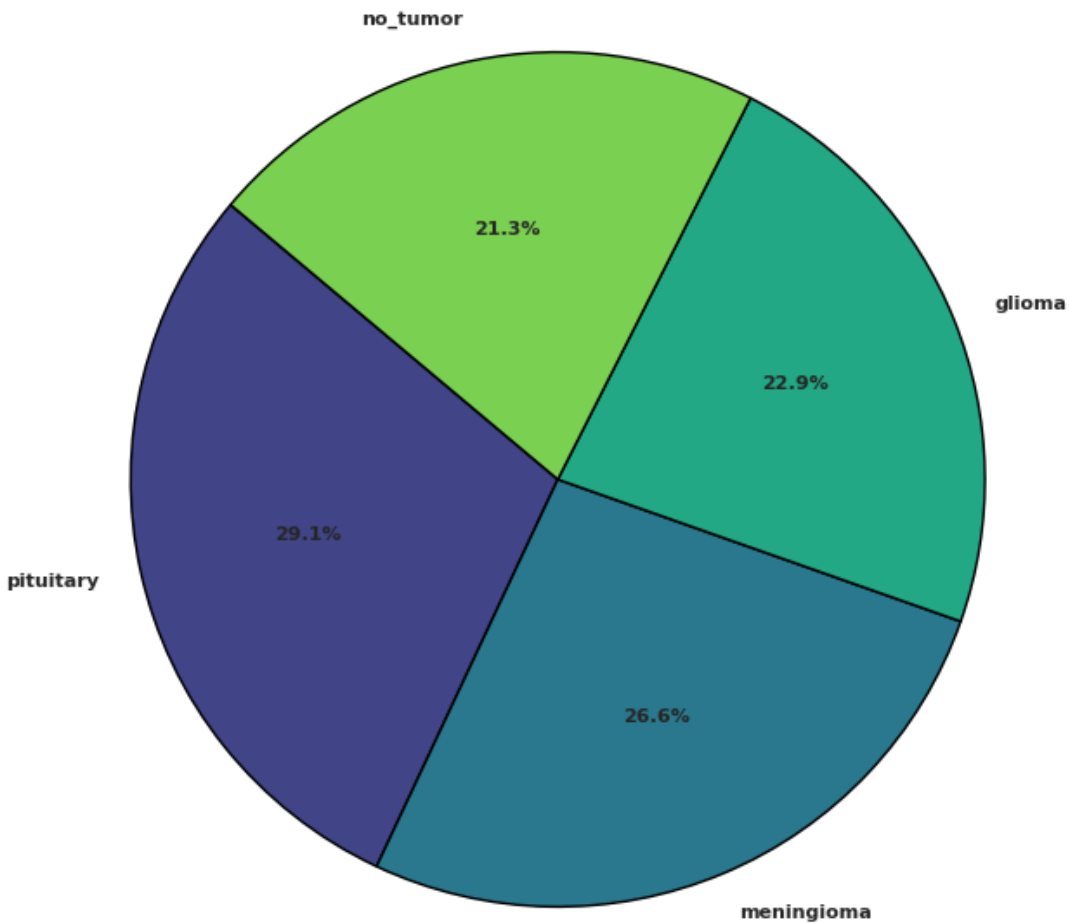
ax.set_title("Distribution of Tumor Types - Pie Chart", fontsize=14,
fontweight='bold')

plt.show()

```



Distribution of Tumor Types - Pie Chart



```
from PIL import Image

num_images = 5

unique_labels = df['label'].unique()

plt.figure(figsize=(15, len(unique_labels) * 3))

for row_idx, label in enumerate(unique_labels):

    label_images = df[df['label'] ==
label].head(num_images)['image_path'].tolist()

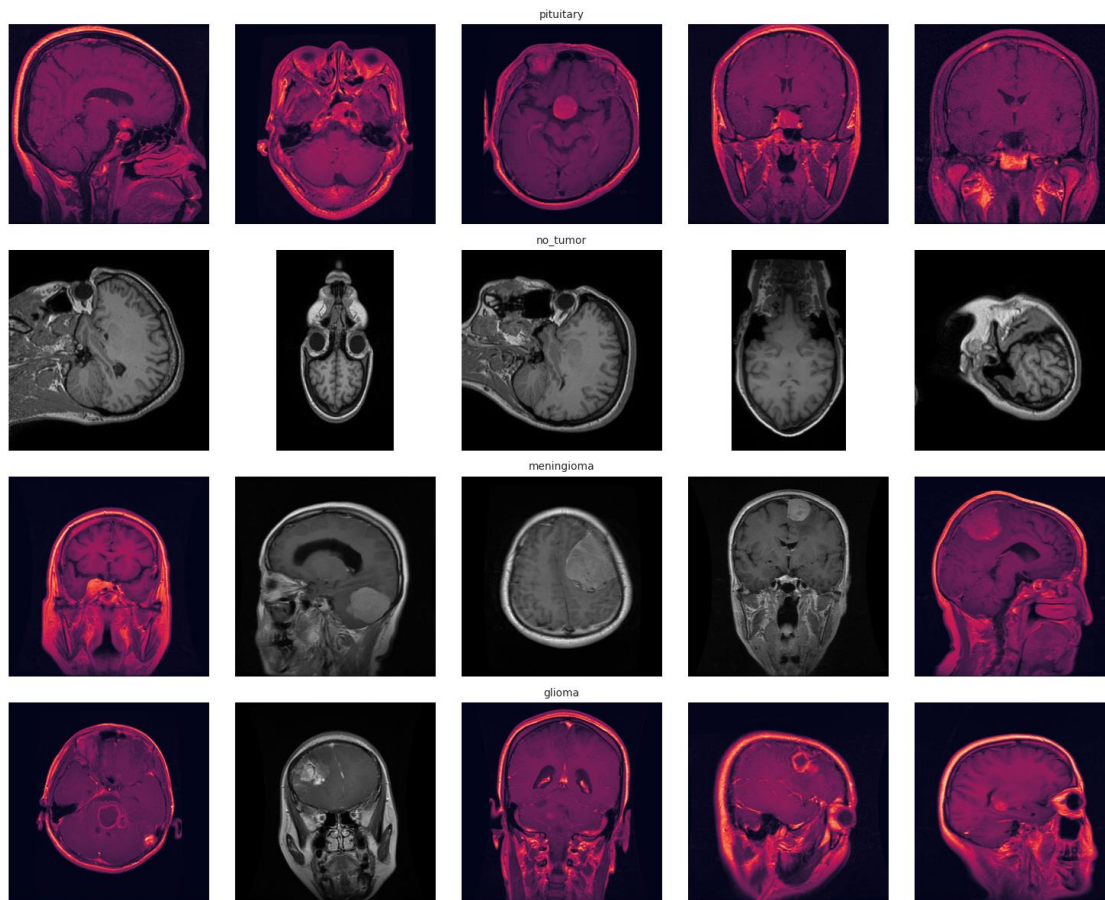
    for col_idx, img_path in enumerate(label_images):
        plt_idx = row_idx * num_images + col_idx + 1
        plt.subplot(len(unique_labels), num_images, plt_idx)
        img = Image.open(img_path)
        plt.imshow(img)
```

```

plt.axis('off')
if col_idx == 2:
    plt.title(label, fontsize=10)

plt.tight_layout()
plt.show()

```



```

import warnings
warnings.filterwarnings('ignore')

import torch
import torch.nn as nn
import torch.optim as optim
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms, models
from PIL import Image
import matplotlib.pyplot as plt
import seaborn as sns
from sklearn.metrics import confusion_matrix
from sklearn.model_selection import train_test_split

train_df, val_df = train_test_split(df, test_size=0.2, stratify=df['label'],
random_state=42)

max_samples = train_df['label'].value_counts().max()

```

```

balanced_train_df = train_df.groupby('label', group_keys=False).apply(
    lambda x: x.sample(n=max_samples, replace=True, random_state=42)
).reset_index(drop=True)

balanced_train_df = balanced_train_df[['image_path', 'label']]

import torch
import torch.nn as nn
import torch.nn.functional as F
import timm
from torch.utils.data import Dataset, DataLoader
from torchvision import transforms
from PIL import Image
from sklearn.metrics import classification_report

class BrainTumorDataset(Dataset):
    def __init__(self, df, transform=None):
        self.df = df
        self.transform = transform
        self.label_map = {label: i for i, label in
            enumerate(sorted(df['label'].unique()))}

    def __len__(self):
        return len(self.df)

    def __getitem__(self, idx):
        img_path = self.df.iloc[idx]['image_path']
        label = self.label_map[self.df.iloc[idx]['label']]
        image = Image.open(img_path).convert('RGB')
        if self.transform:
            image = self.transform(image)
        return image, label

def bjoerck_orthogonalize(w, iters=10):
    with torch.no_grad():
        shape = w.shape
        w_mat = w.view(shape[0], -1)
        a, b = w_mat.shape
        transpose = a < b
        if transpose: w_mat = w_mat.T
        s = torch.linalg.matrix_norm(w_mat, ord=2)
        w_mat = w_mat / (s + 1e-6)
        for _ in range(iters):
            w_mat = 1.5 * w_mat - 0.5 * w_mat @ (w_mat.T @ w_mat)
        if transpose: w_mat = w_mat.T
        return w_mat.view(shape)

class PretrainedViTFOTON(nn.Module):
    def __init__(self, num_classes=4):
        super().__init__()
        self.vit = timm.create_model('vit_tiny_patch16_224', pretrained=True)

```

```

        self.embed_dim = self.vit.embed_dim
        self.classifier = nn.Linear(self.embed_dim, num_classes, bias=False)
        self.classifier.weight.data =
bjorck_orthogonalize(self.classifier.weight.data)

```

```

def forward(self, x):
    x = self.vit.patch_embed(x)
    x = self.vit._pos_embed(x)
    for block in self.vit.blocks:
        x = block(x)
    x = self.vit.norm(x)
    cls_token = x[:, 0]
    return self.classifier(cls_token), cls_token

```

```

transform_pretrained = transforms.Compose([
    transforms.Resize((224, 224)),
    transforms.ToTensor(),
    transforms.Normalize(mean=[0.5, 0.5, 0.5], std=[0.5, 0.5, 0.5]),
])

```

```

train_dataset = BrainTumorDataset(balanced_train_df,
transform=transform_pretrained)
val_dataset = BrainTumorDataset(val_df, transform=transform_pretrained)

```

```

train_loader = DataLoader(train_dataset, batch_size=16, shuffle=True)
val_loader = DataLoader(val_dataset, batch_size=16, shuffle=False)

```

```

device = torch.device('cuda' if torch.cuda.is_available() else 'cpu')
model = PretrainedViTFOTON(num_classes=4).to(device)
optimizer = torch.optim.Adam(model.vit.parameters(), lr=1e-5)
eta = 0.0005
epochs = 25
criterion = nn.CrossEntropyLoss()

```

```

for epoch in range(epochs):
    model.train()
    epoch_loss = 0
    for x, y in train_loader:
        x, y = x.to(device), y.to(device)

        optimizer.zero_grad()
        y_hat, cls_features = model(x)
        loss = criterion(y_hat, y)
        loss.backward()
        optimizer.step()

    epoch_loss += loss.item()

    with torch.no_grad():
        e = torch.softmax(y_hat, dim=1) - F.one_hot(y, 4).float()
        model.classifier.weight.data += -eta * (e.T @ cls_features) /

```

```

x.size(0)
        model.classifier.weight.data =
bjorck_orthogonalize(model.classifier.weight.data)

    print(f'Epoch {epoch+1}/{epochs}, Loss:
{epoch_loss/len(train_loader):.4f}')

model.eval()
y_true, y_pred = [], []
with torch.no_grad():
    for x, y in val_loader:
        x, y = x.to(device), y.to(device)
        out, _ = model(x)
        y_pred.extend(out.argmax(dim=1).cpu().numpy())
        y_true.extend(y.cpu().numpy())

target_names = sorted(df['label'].unique().tolist())
print(classification_report(y_true, y_pred, target_names=target_names))

```

```

Epoch 1/25, Loss: 0.3532
Epoch 2/25, Loss: 0.0541
Epoch 3/25, Loss: 0.0114
Epoch 4/25, Loss: 0.0039
Epoch 5/25, Loss: 0.0014
Epoch 6/25, Loss: 0.0009
Epoch 7/25, Loss: 0.0006
Epoch 8/25, Loss: 0.0005
Epoch 9/25, Loss: 0.0004
Epoch 10/25, Loss: 0.0003
Epoch 11/25, Loss: 0.0002
Epoch 12/25, Loss: 0.0002
Epoch 13/25, Loss: 0.0001
Epoch 14/25, Loss: 0.0001
Epoch 15/25, Loss: 0.0001
Epoch 16/25, Loss: 0.0001
Epoch 17/25, Loss: 0.0001
Epoch 18/25, Loss: 0.0001
Epoch 19/25, Loss: 0.0000
Epoch 20/25, Loss: 0.0000
Epoch 21/25, Loss: 0.0000
Epoch 22/25, Loss: 0.0000
Epoch 23/25, Loss: 0.0000
Epoch 24/25, Loss: 0.0000
Epoch 25/25, Loss: 0.0000

```

	precision	recall	f1-score	support
glioma	0.98	0.97	0.97	229
meningioma	0.96	0.91	0.93	266
no_tumor	0.97	1.00	0.98	213
pituitary	0.95	0.98	0.97	292
accuracy			0.96	1000

macro avg	0.96	0.96	0.96	1000
weighted avg	0.96	0.96	0.96	1000